

SmartMan: 基于 QWen 的 bash 命令生成工具

屈阳

李孟霖

刘治权

赵浦渊

June 24, 2026

研究背景

▪ 传统工具的局限

- 传统 Linux 帮助工具（如 man、tldr）核心支持“依据工具名查询具体用法”。
- 当面对模糊的自然语言需求时（如“查找近 3 天修改过的 sh 文件并统计行数”），用户往往无从下手。

▪ 云端大模型“大材小用”

- 依赖闭源或云端旗舰模型存在“杀鸡用牛刀”的资源浪费。
- 严重依赖网络连接，无法在高离线环境、校园内网或敏感生产宿主机中直接使用，且存在数据隐私泄露风险。

UNIX 哲学: “Do one job and do it right.”

- **设计思路**

- 采用轻量化开源基座模型 + 参数高效微调 (PEFT/LoRA)。
- 打造全本地端侧运行、面向 NL2Bash 任务高度垂直的 AI 助手。

- **SmartMan 的核心目标**

- **系统资源**: 极低的显存与 CPU 占用, 消费级/办公级硬件轻松平替。
- **响应速度**: 首字延迟 (TTFT) 与吞吐速度优于跨网连接的云端大模型浏览器网页, 实现真正的“即搜即用”。

数据与预处理

基座模型 (Base Model) Qwen2.5-1.5B-Instruct

- **规模与效率：**1.5B 轻量级参数，兼顾端侧推理速度与显存友好度。
- **能力对齐：**原生具备强大的中英文代码理解与指令遵循能力。

核心数据集 (Dataset) NL2Bash 语料库¹

- **数据规模：**精选总计约 10k 条高效「自然语言-Bash 命令」对。
- **任务目标：**覆盖文件管理、文本处理、系统监控等高频 Linux 运维场景。

¹Lin et al. Program Synthesis from Natural Language Using RNNs. 从自然语言到 Bash. 2017.

数据划分与规整

按照 8 : 1 : 1 的比例严格划分训练集、验证集和测试集，确保评估结果的泛化性。

ChatML 格式对齐与 System Prompt 约束

全量转换为标准 ChatML 格式 ('jsonl')，在微调阶段显式注入高约束力的系统提示词：

```
"role": "system", "content": " 你是一个 Linux 专家，请根据用户的描述，  
输出对应的 Bash 命令。只输出命令本身，不要任何解释。 "
```

- **核心目的：**强制约束模型输出边界，抑制小模型易出现的幻觉与解释性冗余。
- **特殊 Token 维护：**统一将 pad_token 桥接至 eos_token (<|im_end|>)，避免生成截断与死循环。

技术路线：LoRA 与量化的参数空间

- **基座模型量化 (Quantization)**
 - 原生半精度: `bf16` (保持混合精度训练稳定性)
 - 高级量化: BitsAndBytes 4-bit (NF4) 兼顾精度与显存
 - 常规整型: `int4` / `int8` 极致压缩显存上限
- **SFT 训练策略 (Training Precision)**
 - `bf16` 纯净训练 vs `fp16` 混合精度训练切换
- **LoRA 超参数拓扑 (LoRA Matrix Topology)**
 - 本征秩 (Rank r): 8, 16, 32 梯度阶梯设计
 - 缩放因子 (α): 固定比例控制 $\alpha = 2 \times r$
 - 挂载目标层 (Target Modules):
 - 极致轻量: 仅 `q_proj`, `v_proj`
 - 标准全包: 仅注意力层 (`q`, `k`, `v`, `o`)
 - 全面覆盖: 所有线性层 (All Linear)

排列组合痛点

硬编码脚本会导致样板代码爆炸，亟需工程化解耦。

技术路线：超参数组合的“爆炸”与 quicktune 框架

微调参数空间的挑战

在探索最佳微调策略时，我们需要面对大量参数的排列组合：

- **模型加载**：bf16, int8, bnb/4bit
- **LoRA 策略**：仅 Q/V 层、仅 Attention、all_linear
- **SFT 参数**：学习率、Epoch 衰减、packing

痛点：将上述变量硬编码到主脚本或打平塞入 YAML 会导致配置极其臃肿，且丧失静态类型检查（LSP）的支持。

quicktune 目录拓扑

为此，我们自研了高度解耦的微调框架：

```
src/quicktune/  
  core/                # 核心调度枢纽  
    __init__.py  
    manager.py         # 注册中心 (Registry)  
  recipes/            # 原子化配置插件  
    lora/              # LoRA 拓扑结构库  
      qwen.py  
  models/             # 量化与基座加载库  
    bf16.py , bnb.py , int8.py  
  sft/                # 训练超参策略库  
    bf16.py , fp16.py
```

quicktune 核心机制：Registry 注册表模式

设计哲学：高内聚与动态发现

摒弃传统的 import 硬编码，采用工业界标准的 **Decorator (装饰器)** 模式，在模块导入时自动收集所有配置“图纸”。

核心注册中心 (core/manager.py)

```
class TuneRegistry:
    def __init__(self) -> None:
        # 维护三大全局字典 (Zoo)
        self.model_zoo: Dict[str, ModelBuilder] = {}
        self.lora_zoo: Dict[str, LoraBuilder] = {}
        self.sft_zoo: Dict[str, SFTBuilder] = {}

    def register_lora(self, name: str):
        def wrapper(builder: LoraBuilder) -> LoraBuilder:
            if name in self.lora_zoo:
                raise ValueError(f"Lora config {name} is already registered")
            self.lora_zoo[name] = builder
            return builder
```

quicktune 核心机制：插件化 Recipes 与配置驱动

制定严格的命名空间规范 (< 架构或精度 >/< 风格 >)

通过字典透传 (**kwargs) 保证灵活性，使得 Pipeline 入口只需极简的 YAML 即可动态组装出复杂的微调管线。

LoRA 插件示例

(recipes/lora/qwen.py)

```
@registry.register_lora("qwen/minimal")
def build_lora_minimal(r: int=8, **kwargs):
    """ 最节省参数的方案，仅更新  $Q$  和  $V$  """
    return LoraConfig(
        r=r, lora_alpha=2*r,
        target_modules=["q_proj", "v_proj"],
        bias="none", task_type="CAUSAL_LM"
    )

@registry.register_lora("qwen/all_linear")
def build_lora_all_linear(r: int=16, **kwargs):
    return LoraConfig(
        r=r, lora_alpha=2*r,
        target_modules=["q_proj", "v_proj", "o_proj",
            "gate_proj", "up_proj", ...],
```

模型量化插件

(recipes/models/bnb.py)

```
@registry.register_model("bnb/4bit")
def build_model_4bit(model_id: str):
    """ 标准的 QLoRA 4-bit 加载配置 """
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
    )
    return AutoModelForCausalLM.from_pretrained(
        model_id,
        quantization_config=bnb_config,
        device_map="auto"
    )
```

评估指标 & Benchmark

在 NL2Bash（自然语言转 Bash 命令）任务中，单一的指标难以全面衡量模型生成的质量。

我们的评估框架从**序列绝对匹配**、**词级重合度**、**语法正确性**、**字面相似度**以及**长度偏置**五个维度进行全方位评估：

字词与序列层级

- **S-EM**: 严格完全匹配率
- **BLEU**: 词级 N-gram 重合度
- **Soft-EM_F1**: 自由词频 F1 分数

语法、字面与统计层级

- **Syntax_Pass**: Bash 语法通过率
- **Edit_Sim**: 字符级编辑相似度
- **Len_Ratio**: 预测序列长度比

S-EM (Strict Exact Match) —严格完全匹配

- **定义：**评估模型生成的 Bash 命令在切词（Tokenization）后，是否与真实标签（Ground Truth）**完全一致**（包括顺序与内容）。
- **预处理逻辑：**
 1. 清除首尾空格，将连续的多空格合并为单空格。
 2. 将所有双引号 `""` 替换为单引号 `''` 以消除引用风格差异。
 3. 使用 `shlex.split()` 进行符合标准 Shell 语义的切词。
- **数学表达：**

$$\text{S-EM} = \begin{cases} 1, & \text{if Tokens}_{\text{pred}} = \text{Tokens}_{\text{ref}} \\ 0, & \text{otherwise} \end{cases}$$

- **特点：**最严苛的指标。即使语义等价，若指令参数顺序不同或使用了不同的同义命令（如 `g++` vs `gcc`），也会被判为 0 分。

Soft-EM_F1 —词级 F1 分数（弱完全匹配）

- **定义：**忽略 Token 的先后顺序，仅关注预测集合与真值集合在**词频与内容**上的交集大小。通过计算精确率（Precision）和召回率（Recall）的调和平均值求得。
- **匹配机制：**使用状态数组对真值进行打标（去重匹配计数），准确处理重复 Token 的情况。
- **计算公式：**

$$\text{Precision} = \frac{\text{Common Tokens}}{\text{Len}(\text{Tokens}_{\text{pred}})}, \quad \text{Recall} = \frac{\text{Common Tokens}}{\text{Len}(\text{Tokens}_{\text{ref}})}$$

$$\text{Soft-EM_F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **适用场景：**非常适合评估允许参数无序调换的命令。例如：ls -l -a 与 ls -a -l 在本指标下可获得 100% 的满分。

BLEU (Bilingual Evaluation Understudy) —词级重合度

- **定义：**衡量预测命令与真实命令之间 N-gram 元组的重合百分比。常用于机器翻译和代码生成评测。
- **实现细节：**基于 `nltk.translate.bleu_score` 实现。
 - **输入：**清洗并切词后的 Token 列表。
 - **平滑函数：**采用了 `SmoothingFunction().method4`。
- **平滑意义：**由于 Bash 命令通常较短（N-gram 匹配极易因某个词不匹配而在高阶处归零），Method 4 通过对没有匹配上的高阶 N-gram 赋予一个微小的虚拟计数，避免了短文本下 BLEU 得分陡降为 0 的问题。
- **分值区间：**0.0 ~ 100.0（最终结果乘以 100 转化为百分比）。

- **定义：**独立于真实标签，仅检验模型生成的代码是否是一个**合法的 Bash 脚本**。
- **检测核心：**调用 Python 的 `bashlex` 库对预测出的文本进行抽象语法树（AST）解析。
- **判定逻辑：**
 - **成功 (1)：**解析无误，或触发 `NotImplementedError`（属于 `bashlex` 自身局限，不代表语法错误）。
 - **失败 (0)：**抛出 `bash_errors.ParsingError` 或其他解析异常（如括号未闭合、管道符非法使用等）。
- **指标价值：**NL2Bash 的关键护栏指标。即使逻辑与真值稍有偏差，高语法通过率也说明模型具备基本的代码成型能力。

Edit_Sim (编辑相似度) 与 Len_Ratio (长度比)

5. Edit_Sim — 字符级字面相似度

- **原理：**利用 Python 内置 `difflib.SequenceMatcher` 计算字符维度的相似比例。
- **公式：** $\text{Ratio} = \frac{2 \cdot M}{T}$ （其中 M 为匹配字符数， T 为两文本总字符数）。
- **作用：**能够敏锐发现拼写错误。如将 `grep` 错拼为 `gerp` 时，词级指标全挂，但编辑相似度依然处于高位，能保留局部正确的语义评价。

6. Len_Ratio — 预测序列长度比

- **公式：**

$$\text{Len_Ratio} = \frac{\text{Len}(\text{Tokens}_{\text{pred}})}{\max(1, \text{Len}(\text{Tokens}_{\text{ref}}))}$$

- **作用：**监控大模型的“幻觉”与“复读机”倾向。如果该值远大于 100%，说明模型倾向于过度输出或陷入死循环；远小于 100% 则说明生成发生了解码截断。

输出结果 DataFrame 字段映射指南

每次评估完毕后，系统将自动汇总以下字段并排序落盘：

字段名称	对应指标/配置	度量形式/作用
quant_config	量化与模型架构配置项	分组基准
lora_name	LoRA 微调权重名称	分组基准
S-EM	严格完全匹配率 (Strict EM)	0% ~ 100% (主排序依据)
BLEU	句子级平滑 BLEU 分数	0% ~ 100%
Syntax_Pass	Bash 语法解析通过率	0% ~ 100%
Soft-EM_F1	词级无序 F1 值 (Soft EM)	0% ~ 100%
Edit_Sim	字符级编辑距离相似度	0% ~ 100%
Len_Ratio	词数长度比 (Prediction / GT)	百分比 (理想值为 100%)

Table 1. LoRA 与量化配置在 NL2Bash 任务上的性能对比

Quant	LoRA Name	S-EM (%)	BLEU	Syn-Pass (%)	Soft- F_1 (%)	Edit-Sim	Len-Ratio (%)
bf16	qwenInstruct_short0501_0107	14.91	30.78	99.05	61.25	72.51	112.34
bnb/4bit	qwenInstruct_short0501_2322	14.35	30.34	98.41	60.91	72.38	112.80
bnb/4bit	qwenInstruct_short_larger_r0502_1501	14.27	29.31	98.97	60.45	71.65	109.68
bnb/4bit	qwenInstruct_larger_r0502_0011	13.56	27.95	99.44	59.39	71.03	110.08
bf16	qwenInstruct_short0501_2322	13.48	27.95	98.89	59.24	70.91	113.12
bf16	qwenInstruct0429_1852	13.24	29.36	98.97	60.09	71.84	111.90
bf16	qwenInstruct_short_larger_r0502_1501	12.21	26.57	99.13	59.00	70.94	111.88
bf16	qwenInstruct_larger_r0502_0011	11.50	25.96	99.05	58.30	70.51	110.00
bnb/4bit	qwenInstruct_short0501_0107	9.83	25.66	97.86	56.88	68.94	121.50
bf16	qwenInstruct_smaller_r0502_1424	9.36	22.37	99.05	53.62	67.04	113.73
bnb/4bit	qwenInstruct_smaller_r0502_1424	8.88	22.79	98.97	54.44	67.17	116.71
bnb/4bit	qwenInstruct0429_1852	8.88	24.94	99.05	56.26	67.81	123.57
bf16	qwen0429_1607	7.30	19.19	98.18	49.28	62.34	138.48
bf16	Base_Model	6.03	16.46	98.57	43.65	59.79	115.70
bnb/4bit	qwen0429_1607	4.04	14.58	97.54	41.49	58.29	156.68
bnb/4bit	Base_Model	1.51	12.10	98.41	35.84	54.37	136.73

实验结果分析：LoRA 微调与基座对比

■ 微调带来质的飞跃 (对比 Base Model)

- 未经微调的 Base_Model (bf16) 的严格完全匹配率 (S-EM) 仅为 **6.03%**，BLEU 为 **16.46**。
- 微调后的最优 Checkpoint 实现了 S-EM **14.91%** (相对提升 **147.3%**)，BLEU 翻倍至 **30.78**，证明了针对性 SFT 的必要性。

■ 指令对齐基座 (Instruct) 优势显著

- 基于 qwenInstruct 基座的微调表现 ($S-EM \geq 8.88\%$) 普遍远超纯基准基座的微调 (qwen0429 仅 7.30%)。
- **结论**：通用指令对齐赋予了模型更好的任务边界依从性，从而大幅提升了下游代码生成任务的微调上限。

■ 幻觉与冗余输出的有效控制

- Base_Model 表现出严重的“复读”或“解释”倾向，导致其长度比 (Len-Ratio) 严重偏高 (**136.73%**)。
- 最优 Recipe 成功将 Len-Ratio 压制在 **109% ~ 112%**，输出形式更逼近真实脚本。

实验结果分析：量化损耗与生成鲁棒性

4-bit 量化表现出极高的「抗损性」

- **差距微弱**：bf16 顶峰 S-EM 为 **14.91%**，而 bnb/4bit 顶峰为 **14.35%**，性能损耗仅为 **0.56%**。
- **工程落地价值**：在大幅压缩显存占用的前提下，4-bit 量化几乎实现了对语义泛化能力的**无损压缩**。

语法通过率 (Syn-Pass) 形成安全护栏

- 纵观所有测试组合，模型的语法通过率表现极为稳健，均维持在 **97.5% ~ 99.5%** 的超高区间。
- **结论**：评测的主要瓶颈不在于“写出合法的 Bash 语法”，而在于“精准切中用户意图的参数排列 (S-EM)”。

最优模型配置总结 (qwenInstruct_short)

依托 Qwen2.5-1.5B-Instruct 为基座，采用 bnb/4bit 基础量化 + 全线性层 LoRA (r=16)，在 $2e-4$ 学习率下历经 5 个 Epoch 的 Packing SFT 训练，取得了兼顾显存经济性与生成性能的帕累托最优解。

最终落地产品 smartman-cli

- 三款不同量化程度的 gguf
- llamafire 通用推理后端
- Rust 编写的轻量化 REPL 客户端

谢谢大家！